

COMP3242/6242: Deep Learning

Week 11 — Generative AI

Stephen Gould

`stephen.gould@anu.edu.au`

Australian National University

Semester 1, 2026

Overview

- ▶ generative models — learning to sample
- ▶ Gaussian refresher
- ▶ VAEs and ELBO
- ▶ normalising flows
- ▶ diffusion models

Reading and Resources.

- ▶ Kingma and Welling, “Auto-Encoding Variational Bayes”, 2014
- ▶ Papamakarios et al., “Normalizing flows for probabilistic modeling and inference”, 2021
- ▶ Sohl-Dickstein et al., “Deep Unsupervised Learning using Nonequilibrium Thermodynamics”, 2015
- ▶ Ho et al., “Denoising Diffusion Probabilistic Models”, 2020
- ▶ Song et al., “Score-Based Generative Modeling through SDEs”, 2021

Generative Models

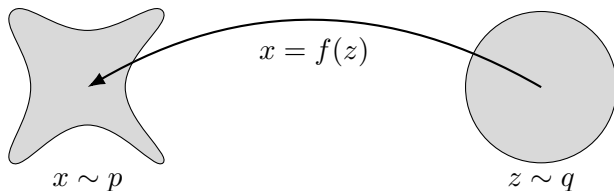
- ▶ generative models learn joint distributions over data
 - ▶ c.f. discriminative models (e.g., classifiers) that learn boundaries between data

Generative Models

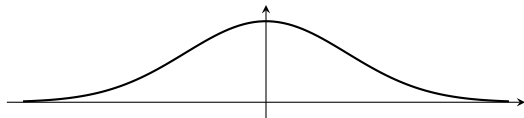
- ▶ generative models learn joint distributions over data
 - ▶ c.f. discriminative models (e.g., classifiers) that learn boundaries between data
- ▶ **why?** allow us sample i.i.d. from some unknown or hard-to-sample distribution, p
- ▶ **how?** learn a mapping f from an easy-to-sample distribution q to p
 - ▶ a good choice for q is the multivariate Gaussian

Generative Models

- ▶ generative models learn joint distributions over data
 - ▶ c.f. discriminative models (e.g., classifiers) that learn boundaries between data
- ▶ **why?** allow us sample i.i.d. from some unknown or hard-to-sample distribution, p
- ▶ **how?** learn a mapping f from an easy-to-sample distribution q to p
 - ▶ a good choice for q is the multivariate Gaussian



Gaussian Refresher

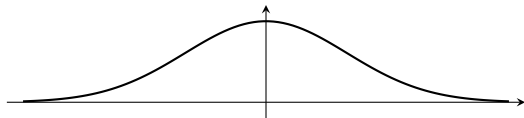


- ▶ multivariate Gaussian with mean $\mu \in \mathbb{R}^n$ and covariance $\Sigma \in \mathbb{R}^{n \times n}$ has form

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$

- ▶ often written $\mathcal{N}(x; \mu, \Sigma)$ or $\mathcal{N}(\mu, \Sigma)$
 - ▶ special case: zero mean, unit covariance, $\mathcal{N}(0, I)$

Gaussian Refresher

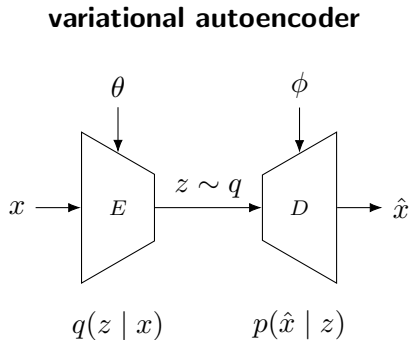
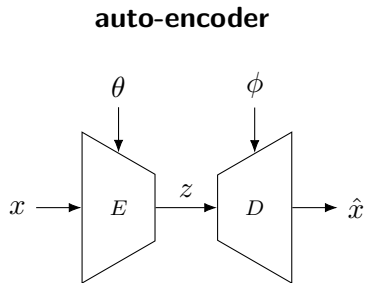


- ▶ multivariate Gaussian with mean $\mu \in \mathbb{R}^n$ and covariance $\Sigma \in \mathbb{R}^{n \times n}$ has form

$$p(x; \mu, \Sigma) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma|^{\frac{1}{2}}} \exp \left(-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right)$$

- ▶ often written $\mathcal{N}(x; \mu, \Sigma)$ or $\mathcal{N}(\mu, \Sigma)$
 - ▶ special case: zero mean, unit covariance, $\mathcal{N}(0, I)$
- ▶ elements of x are independent, $p(x_i, x_j) = p(x_i)p(x_j)$, iff covariance is diagonal
- ▶ conditional $p(x | y)$ of jointly Gaussian variables (x, y) is Gaussian
- ▶ marginal $p(x) = \int p(x, y) dy$ of jointly Gaussian variables (x, y) is Gaussian
- ▶ we write $x \sim \mathcal{N}(\mu, \Sigma)$ to mean x is sampled from Gaussian $\mathcal{N}(\mu, \Sigma)$
 - ▶ equivalently $x = \mu + \Sigma^{1/2} \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$

Variational Autoencoder (VAE)



- ▶ q chosen to be a Gaussian, so E predicts $\mu(x)$ and $\Sigma(x)$ (often diagonal)
- ▶ z is sampled as $z = \mu + \Sigma^{1/2}\epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, which D decodes

Variational Inference

- ▶ our goal is to maximize the marginal log-likelihood of the observed data

$$\log p(x) = \log \left(\int_z p(x, z) \, dz \right)$$

which is intractable

Variational Inference

- ▶ our goal is to maximize the marginal log-likelihood of the observed data

$$\log p(x) = \log \left(\int_z p(x, z) \, dz \right)$$

which is intractable

- ▶ **variational inference** approximates $p(z \mid x)$ with $q(z \mid x)$ from the encoder

$$\begin{aligned} \log p(x) &= \log \left(\int_z p(x, z) \, dz \right) \\ &= \log \left(\int_z q(z \mid x) \frac{p(x, z)}{q(z \mid x)} \, dz \right) \\ &\geq E_{q(z \mid x)} \left[\log \frac{p(x, z)}{q(z \mid x)} \right] && \text{(Jensen's inequality)} \\ &= E_{q(z \mid x)} \left[\log \frac{p(x \mid z)p(z)}{q(z \mid x)} \right] \\ &= E_{q(z \mid x)} [\log p(x \mid z)] - E_{q(z \mid x)} \left[\log \frac{q(z \mid x)}{p(z)} \right] \end{aligned}$$

Evidence Lower Bound (ELBO)

$$\ell^{\text{ELBO}}(\theta, \phi) = \underbrace{E_{q(z|x)} [\log p(x | z; \phi)]}_{\text{reconstruction loss}} - \overbrace{D_{\text{KL}}(q(z | x; \theta) \| p(z))}^{\text{regularization}}$$

- ▶ $q(z | x; \theta) = \mathcal{N}(\mu, \Sigma)$ encoder estimated distribution of z from x
- ▶ $p(x | z; \phi)$ decoder reconstruction of x from z
- ▶ $p(z)$ is the prior distribution of z , i.e., $\mathcal{N}(0, I)$

Evidence Lower Bound (ELBO)

$$\ell^{\text{ELBO}}(\theta, \phi) = \underbrace{E_{q(z|x)} [\log p(x | z; \phi)]}_{\text{reconstruction loss}} - \overbrace{D_{\text{KL}}(q(z | x; \theta) \| p(z))}^{\text{regularization}}$$

- ▶ $q(z | x; \theta) = \mathcal{N}(\mu, \Sigma)$ encoder estimated distribution of z from x
- ▶ $p(x | z; \phi)$ decoder reconstruction of x from z
- ▶ $p(z)$ is the prior distribution of z , i.e., $\mathcal{N}(0, I)$
- ▶ maximizing ELBO maximizes a **lower bound** of the log-likelihood of the data
- ▶ ELBO balances two objectives: good reconstruction and latent space regularization (to allow for smooth sampling)
- ▶ Gaussian assumption simplifies expressions

Training VAEs

initialize θ, ϕ

repeat

$\ell = 0$

▷ zero loss

for each sample x in mini-batch **do**

$\mu, \Sigma = E(x; \theta)$

▷ encoder forward pass

$\epsilon \sim \mathcal{N}(0, I)$

▷ sample noise

$z = \mu + \Sigma^{1/2} \epsilon$

▷ reparameterization trick

$\hat{x} = D(z; \phi)$

▷ decoder forward pass

$\ell \leftarrow \ell + \frac{1}{2} \|\hat{x} - x\|^2 - \frac{1}{2} (\text{tr}(\Sigma) + \|\mu\|^2 - \log \det \Sigma)$

▷ update loss

end for

$\theta \leftarrow \theta - \eta \nabla_{\theta} \ell$

▷ update encoder parameters

$\phi \leftarrow \phi - \eta \nabla_{\phi} \ell$

▷ update decoder parameters

until converged

return θ, ϕ

► often assume diagonal Σ and estimates $\log \sigma$ instead of σ for numerical stability

VAE Extensions

- ▶ **conditional VAEs** extend standard VAEs by conditioning both the encoder and decoder on auxiliary information y , such as class labels, to control generation
- ▶ **vector quantized VAEs** use a discrete latent space rather than continuous, allowing for more expressive reconstructions and avoiding posterior collapse

Normalizing Flows

- ▶ directly learn mapping between distributions

$$x = f(z), \quad z \sim q(z)$$

- ▶ change of variable formula to maintain normalization of probability distribution

$$\log p(x) = \log q(x) - \log \det \frac{\partial f}{\partial z}$$

- ▶ requires invertible f else logdet becomes zero
- ▶ requires expensive computation of Jacobians
 - ▶ later methods simplify this computation
- ▶ for more see MIT course on flow matching and diffusion models
<https://www.youtube.com/playlist?list=PL57nT7tSGAAXwjhDYcxEycx5W7YoSrZyt>

Denoising Diffusion Probabilistic Models (DDPM)

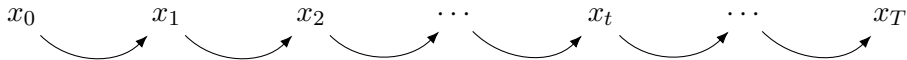
(Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021)

- ▶ **forward diffusion process:** data, $x_0 \in \mathbb{R}^n \rightarrow$ noise, $x_T \in \mathbb{R}^n$
- ▶ **reverse denoising process:** noise, $x_T \in \mathbb{R}^n \rightarrow$ data, $x_0 \in \mathbb{R}^n$

Denoising Diffusion Probabilistic Models (DDPM)

(Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021)

- ▶ **forward diffusion process:** data, $x_0 \in \mathbb{R}^n \rightarrow$ noise, $x_T \in \mathbb{R}^n$
- ▶ **reverse denoising process:** noise, $x_T \in \mathbb{R}^n \rightarrow$ data, $x_0 \in \mathbb{R}^n$



$$q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

- ▶ where β_t is the variance of the noise at step t

Denoising Diffusion Probabilistic Models (DDPM)

(Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2021)

- ▶ **forward diffusion process:** data, $x_0 \in \mathbb{R}^n \rightarrow$ noise, $x_T \in \mathbb{R}^n$
- ▶ **reverse denoising process:** noise, $x_T \in \mathbb{R}^n \rightarrow$ data, $x_0 \in \mathbb{R}^n$



x_0 x_1 x_2 $\dots$ x_t $\dots$ x_T

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

- ▶ where $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ captures the accumulated noise

Forward Recursion with Reparameterization Trick

$$x_t \sim \mathcal{N}(\sqrt{1 - \beta_t}x_{t-1}, \beta_t I)$$

$$x_t = \sqrt{1 - \beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon_t,$$

$$\epsilon_t \sim \mathcal{N}(0, I)$$

$$= \sqrt{\alpha_t}x_{t-1} + \sqrt{1 - \alpha_t}\epsilon_t$$

$$= \sqrt{\alpha_t} \left(\sqrt{\alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_{t-1}}\epsilon_{t-1} \right) + \sqrt{1 - \alpha_t}\epsilon_t$$

$$= \sqrt{\alpha_t \alpha_{t-1}}x_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}}\epsilon$$

$\vdots$

$$= \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon,$$

$$\epsilon \sim \mathcal{N}(0, I)$$

- where ϵ on the fifth line comes from the fact that if $X \sim \mathcal{N}(0, \sigma_X^2 I)$ and $Y \sim \mathcal{N}(0, \sigma_Y^2 I)$ then $X + Y \sim \mathcal{N}(0, (\sigma_X^2 + \sigma_Y^2)I)$

Diffusion Kernel and Forward Sampling

- ▶ $q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$ is called the **diffusion kernel**
- ▶ defining as above, i.e., $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, is called the **reparameterization trick** and allows
 - ▶ jumping steps in the forward process
 - ▶ back-propagation (as we will see later for learning reverse sampling)
- ▶ β_t (noise schedule) chosen so that $\bar{\alpha}_T \rightarrow 0$, i.e., $q(x_T | x_0) \approx \mathcal{N}(0, I)$

Diffusion Kernel and Forward Sampling

- ▶ $q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I)$ is called the **diffusion kernel**
- ▶ defining as above, i.e., $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$, is called the **reparameterization trick** and allows
 - ▶ jumping steps in the forward process
 - ▶ back-propagation (as we will see later for learning reverse sampling)
- ▶ β_t (noise schedule) chosen so that $\bar{\alpha}_T \rightarrow 0$, i.e., $q(x_T | x_0) \approx \mathcal{N}(0, I)$
- ▶ note that we can't express $q(x_t) = \int q(x_0)q(x_t | x_0)dx_0$ in closed-form
- ▶ but we can generate samples from it as follows
 - ▶ first sample from the data distribution $x_0 \sim q(x_0)$
 - ▶ then sample from the diffusion kernel, $x_t \sim q(x_t | x_0)$

Data Generation



► standard generative (or reverse sampling) process:

- sample $x_T \sim N(0, I)$
- iteratively sample $x_{t-1} \sim q(x_{t-1} | x_t)$

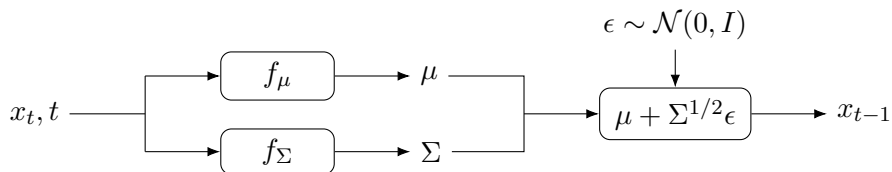
Data Generation



- ▶ standard generative (or reverse sampling) process:
 - ▶ sample $x_T \sim N(0, I)$
 - ▶ iteratively sample $x_{t-1} \sim q(x_{t-1} | x_t)$
- ▶ **problem:** $q(x_{t-1} | x_t) \propto q(x_{t-1})q(x_t | x_{t-1})$ is intractable
 - ▶ note that $q(x_{t-1} | x_t, x_0)$ is tractable, a property that we will use later
- ▶ can we approximate $q(x_{t-1} | x_t)$?
 - ▶ yes, as long as β_t is small we can use a Gaussian, $\mathcal{N}(f_\mu(x_t, t; \theta), f_\Sigma(x_t, t; \theta))$ where f_μ and f_Σ are learned neural networks

Sampling from $q(x_{t-1} \mid x_t)$

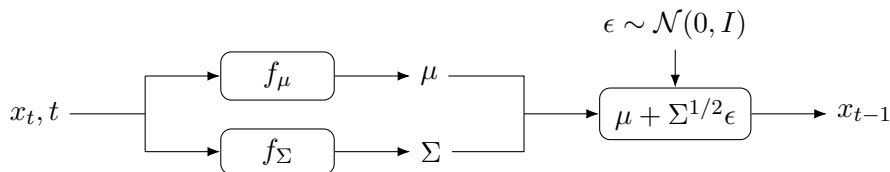
- network to sample x_{t-1} from $\mathcal{N}(f_\mu(x_t, t; \theta), f_\Sigma(x_t, t; \theta))$



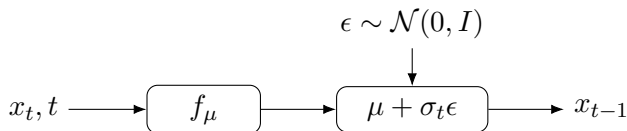
- uses reparameterization trick so that we can back-propagate through μ and Σ

Sampling from $q(x_{t-1} \mid x_t)$

- ▶ network to sample x_{t-1} from $\mathcal{N}(f_\mu(x_t, t; \theta), f_\Sigma(x_t, t; \theta))$



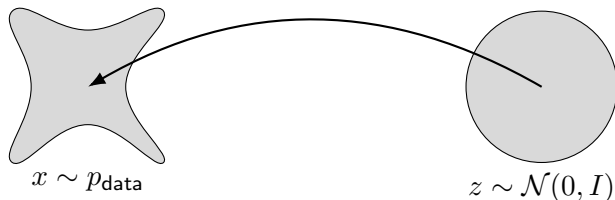
- ▶ uses reparameterization trick so that we can back-propagate through μ and Σ
- ▶ often simplified as $\mathcal{N}(f_\mu(x_t, t; \theta), \sigma_t^2 I)$



- ▶ use a U-net for f_μ (since μ is the same size as x_t)

Summary so Far

- ▶ we would like to generate samples from an unknown data distribution
- ▶ we have defined a diffusion process that takes data samples and produces noise
- ▶ the reverse diffusion process takes noise and produces a sample from the (unknown) data distribution



- ▶ but we need a network f_μ for $q(x_{t-1} \mid x_t)$ in the reverse process
- ▶ to obtain f_μ we need a way to learn a parametrized distribution, $\mathcal{N}(f_\mu(x_t, t; \theta), \sigma_t^2 I)$, to match a data distribution

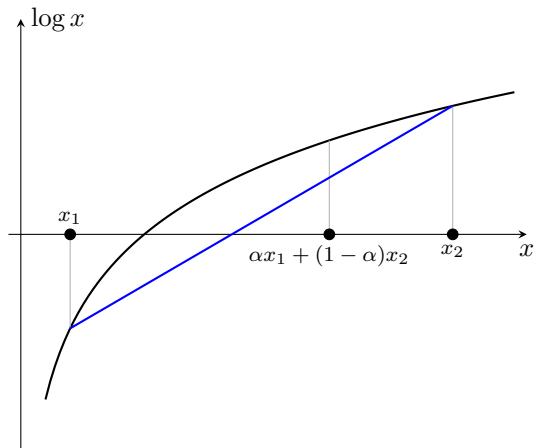
Variational Lower Bound

- ▶ let p be a parametrized distribution that we wish to learn
- ▶ let q be a known/empirical/assumed distribution that is easy to evaluate
- ▶ for brevity write $x_{0:T} \triangleq (x_0, \dots, x_T)$, then

$$\begin{aligned}\log p(x_0; \theta) &= \log \int p(x_{0:T}; \theta) \, dx_{1:T} \\ &= \text{red} \int \text{blue } q(x_{1:T} \mid x_0) \frac{p(x_{0:T}; \theta)}{q(x_{1:T} \mid x_0)} \, dx_{1:T} \\ &\geq \int q(x_{1:T} \mid x_0) \text{red} \left(\frac{p(x_{0:T}; \theta)}{q(x_{1:T} \mid x_0)} \right) \, dx_{1:T} \quad (\text{Jensen's inequality})\end{aligned}$$

Jensen's Inequality

$$\log E[x] \geq E[\log x]$$



Variational Lower Bound (2)

- ▶ let p be a parametrized distribution that we wish to learn
- ▶ let q be a known/empirical/assumed distribution that is easy to evaluate
- ▶ for brevity write $x_{0:T} \triangleq (x_0, \dots, x_T)$, then

$$\begin{aligned}\log p(x_0) &= \log \int p(x_{0:T}) \mathrm{d}x_{1:T} \\ &= \log E_{q(x_{1:T}|x_0)} \left[\frac{p(x_{0:T})}{q(x_{1:T} | x_0)} \right] \\ &\geq E_{q(x_{1:T}|x_0)} \left[\log \left(\frac{p(x_{0:T})}{q(x_{1:T} | x_0)} \right) \right]\end{aligned}$$

- ▶ note that expectations can be approximated as a finite sum over samples

Training Loss

- ▶ minimize the negative log likelihood of the data distribution under the model

$$\begin{aligned} L^{\text{nl}}(\theta) &= - \int q(x_0) \log p(x_0) \, dx_0 \\ &\leq - \int q(x_{0:T}) \log \left(\frac{p(x_{0:T})}{q(x_{1:T} \mid x_0)} \right) \, dx_{0:T} \\ L^{\text{vlb}}(\theta) &= \int q(x_{0:T}) \log \left(\frac{q(x_{1:T} \mid x_0)}{p(x_{0:T})} \right) \, dx_{0:T} \\ &= E_{q(x_{0:T})} \left[\log \left(\frac{q(x_{1:T} \mid x_0)}{p(x_{0:T})} \right) \right] \end{aligned}$$

Training Loss for Gaussians

- ▶ after some algebraic manipulation (Sohl-Dickstein et al., 2015):

$$L^{\text{vlb}} = E_{q(x_{0:T})} \left[\begin{aligned} &D_{\text{KL}}(q(x_T \mid x_0) \parallel p(x_T)) + && (L_T) \\ &\sum_{t=2}^T D_{\text{KL}}(q(x_{t-1} \mid x_t, x_0) \parallel p(x_{t-1} \mid x_t)) + && (L_{t-1}) \\ &-\log p(x_0 \mid x_1) \end{aligned} \right] && (L_0)$$

where $D_{\text{KL}}(P \parallel Q) = E_P \left[\log \frac{P}{Q} \right]$ is the KL-divergence

Training Loss for Gaussians

- ▶ after some algebraic manipulation (Sohl-Dickstein et al., 2015):

$$L^{\text{vlb}} = E_{q(x_{0:T})} \left[\begin{aligned} &D_{\text{KL}}(q(x_T \mid x_0) \parallel p(x_T)) + && (L_T) \\ &\sum_{t=2}^T D_{\text{KL}}(q(x_{t-1} \mid x_t, x_0) \parallel p(x_{t-1} \mid x_t)) + && (L_{t-1}) \\ &-\log p(x_0 \mid x_1) \end{aligned} \right] \quad (L_0)$$

where $D_{\text{KL}}(P \parallel Q) = E_P \left[\log \frac{P}{Q} \right]$ is the KL-divergence

- ▶ observe that
 - ▶ L_T is constant and can be ignored (since $x_T \sim N(0, I)$)
 - ▶ L_{t-1} compares two Gaussians so can be written in closed form
 - ▶ L_0 dealt with as a special case

Computing L_t (for $t = 1, \dots, T - 1$)

we have

$$q(x_{t-1} \mid x_t, x_0) = \frac{q(x_t \mid x_{t-1}, x_0)q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)} \quad (\text{Bayes rule})$$

$$= \mathcal{N}\left(\frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t\right), \tilde{\beta}_t I\right) \quad (\text{algebra})$$

and

$$\begin{aligned} p(x_{t-1} \mid x_t) &= \mathcal{N}(f_\mu(x_t, t; \theta), f_\Sigma(x_t, t; \theta)) \\ &= \mathcal{N}(f_\mu(x_t, t; \theta), \sigma_t^2 I) \end{aligned}$$

to give

$$L_{t-1} = E_{q, \epsilon} \left[\frac{1}{\kappa} \left\| \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t\right) - f_\mu(x_t, t; \theta) \right\|^2 \right] + \text{const.}$$

where κ is a scaling factor involving α_t and σ_t

Simplified Training Loss

Ho et al., (2020)

- ▶ instead of learning f_μ directly, let us write

$$f_\mu(x_t, t; \theta) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} f_\epsilon(x_t, t; \theta) \right)$$

which has the same form as $q(x_{t-1} | x_t, x_0)$

- ▶ that is, learn a network to predict the noise added from x_{t-1} to x_t
- ▶ dropping the scaling factor κ , the simplified loss becomes

$$L^{\text{simplified}}(\theta) = E_{x_0, t, \epsilon_t} \left[\underbrace{\| \epsilon_t - f_\epsilon(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_t, t; \theta) \|^2}_{\text{regression loss}} \right]$$

Training Algorithm

initialize θ

repeat

$$x_0 \sim q(x_0)$$

$$t \sim U(1, T)$$

$$\epsilon \sim \mathcal{N}(0, I)$$

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

$$\ell(\theta) = \|\epsilon - f_\epsilon(x_t, t; \theta)\|^2$$

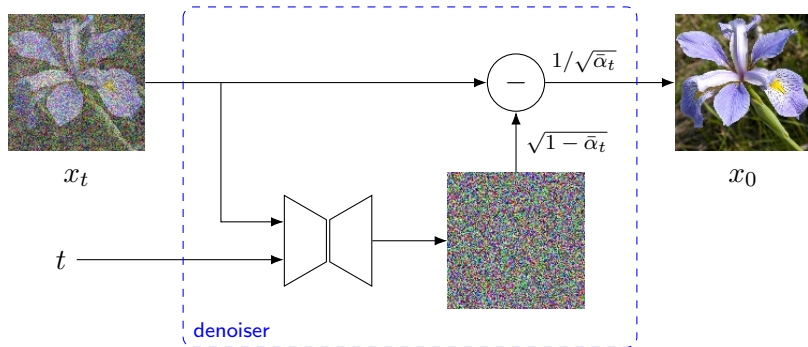
$$\theta \leftarrow \theta - \eta \nabla_\theta \ell(\theta)$$

until converged

return θ

- ▷ sample data
- ▷ sample timestep
- ▷ sample noise
- ▷ forward recursion
- ▷ compute loss
- ▷ update parameters

Training Architecture



Sampling Algorithm

$$x_T \sim \mathcal{N}(0, I)$$

▷ sample noise

for $t = T, \dots, 1$ **do**

if $t > 1$ **then**

$$z \sim \mathcal{N}(0, I)$$

else

$$z = 0$$

end if

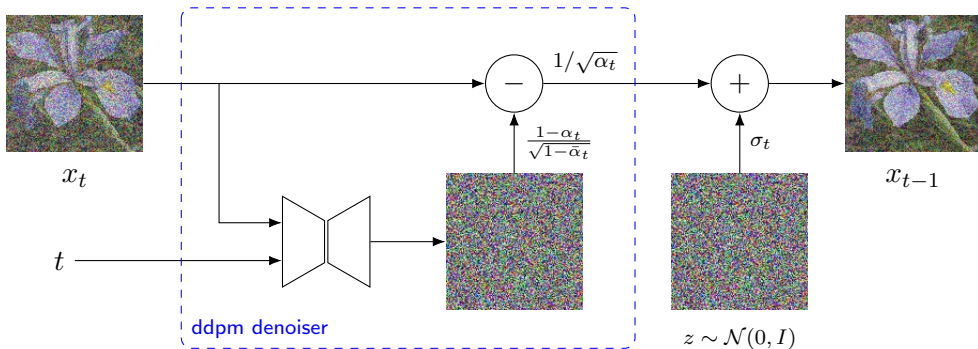
$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} f_\epsilon(x_t, t; \theta) \right) + \sigma_t z$$

▷ denoise

end for

return x_0

Sampling/Generation Architecture



Extensions and Theory

- ▶ we've only just touched the surface on diffusion models
- ▶ theory can be traced back to reverse-time SDEs (Anderson, 1982)
 - ▶ students interested in SDEs can take STAT3006/6060 at ANU
- ▶ there have also been many practical improvements over the past several years
 - ▶ incorporate explicit class information through **guidance** (Dhariwal and Nichol, 2021)

$$f_{\epsilon}(x_t, t; \theta) - w \nabla_{x_t} \log f^{\text{classifier}}(y | x_t)$$

- ▶ classifier-free guidance by learning a conditional noise model, $f_{\epsilon}(x_t, t, y; \theta)$
- ▶ speeding up (e.g., $T = 1000$ steps), e.g., (Nichol and Dhariwal, 2021; Song et al., 2020; Salimans and Ho, 2022)
- ▶ latent diffusion models (Rombach et al., 2022) instead of pixel space
- ▶ tricks for higher resolution and video diffusion

Latent Diffusion Model Architecture

(Rombach et al., 2022)

